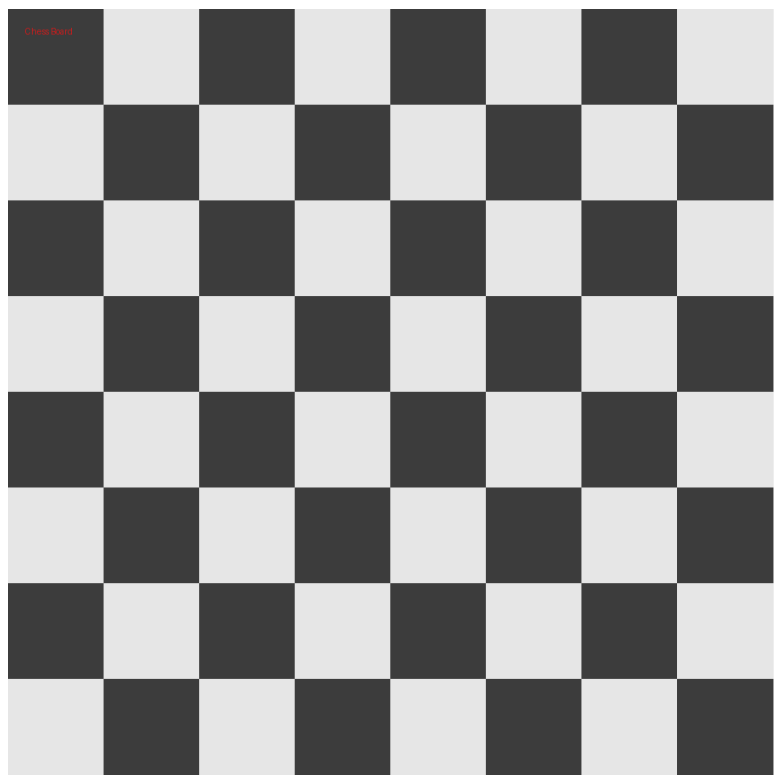
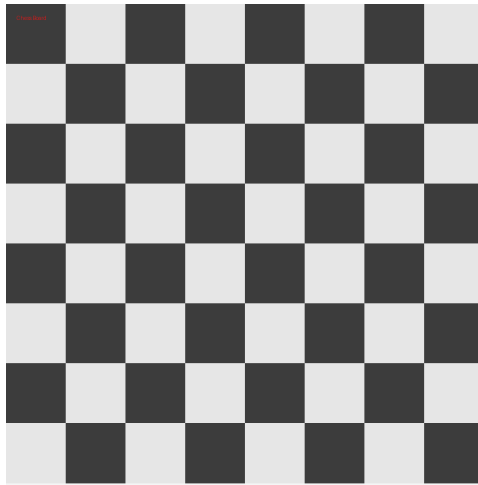


# Python Games: Chess & Tic-Tac-Toe

Two console-based, two-player Python games.



# Chess (chess.py)



```
"""
Simple two-player console Chess game.
Run with: python chess.py

Moves are entered in algebraic coordinate form, e.g.:
    e2 e4
Type 'quit' to exit.

Supports: normal moves, captures, pawn double-step, pawn promotion (auto to Queen),
basic castling, en passant. Does NOT enforce check/checkmate (you can move into check) -
it's meant as a casual two-player board, not a full rules engine.
"""

BOARD_SIZE = 8

WHITE, BLACK = "white", "black"

PIECE_SYMBOLS = {
    ("white", "P"): "♙", ("white", "N"): "♘", ("white", "B"): "♚",
    ("white", "R"): "♖", ("white", "Q"): "♑", ("white", "K"): "♔",
    ("black", "P"): "♟", ("black", "N"): "♞", ("black", "B"): "♝",
    ("black", "R"): "♞", ("black", "Q"): "♛", ("black", "K"): "♚",
}

class Piece:
    def __init__(self, color, kind):
        self.color = color
        self.kind = kind # P, N, B, R, Q, K
        self.has_moved = False

    def symbol(self):
        return PIECE_SYMBOLS[(self.color, self.kind)]

def make_initial_board():
    board = [[None] * BOARD_SIZE for _ in range(BOARD_SIZE)]
    order = ["R", "N", "B", "Q", "K", "B", "N", "R"]
    for col in range(BOARD_SIZE):
        board[0][col] = Piece(BLACK, order[col])
        board[1][col] = Piece(BLACK, "P")
        board[6][col] = Piece(WHITE, "P")
        board[7][col] = Piece(WHITE, order[col])
    return board

def parse_square(s):
    s = s.strip().lower()
    if len(s) != 2 or s[0] not in "abcdefgh" or s[1] not in "12345678":
        return None
    col = ord(s[0]) - ord("a")
    row = 8 - int(s[1])
```

```

    return row, col

def square_name(row, col):
    return "abcdefgh"[col] + str(8 - row)

def print_board(board, last_move=None):
    print()
    print("  a b c d e f g h")
    for row in range(BOARD_SIZE):
        line = f"{8 - row} "
        for col in range(BOARD_SIZE):
            piece = board[row][col]
            cell = piece.symbol() if piece else "."
            line += f"{cell} "
        print(line + f" {8 - row}")
    print("  a b c d e f g h")
    print()

def in_bounds(row, col):
    return 0 <= row < BOARD_SIZE and 0 <= col < BOARD_SIZE

def get_piece_moves(board, row, col, en_passant_target):
    """Return list of (row, col) candidate destination squares for the piece at (row,col),
    ignoring whether the move would leave own king in check."""
    piece = board[row][col]
    if piece is None:
        return []
    color = piece.color
    kind = piece.kind
    moves = []

    def enemy(r, c):
        p = board[r][c]
        return p is not None and p.color != color

    def empty(r, c):
        return board[r][c] is None

    if kind == "P":
        direction = -1 if color == WHITE else 1
        start_row = 6 if color == WHITE else 1
        # forward
        r1 = row + direction
        if in_bounds(r1, col) and empty(r1, col):
            moves.append((r1, col))
            r2 = row + 2 * direction
            if row == start_row and empty(r2, col):
                moves.append((r2, col))
        # captures
        for dc in (-1, 1):
            r1, c1 = row + direction, col + dc
            if in_bounds(r1, c1):
                if enemy(r1, c1):
                    moves.append((r1, c1))
                elif en_passant_target == (r1, c1):
                    moves.append((r1, c1))

    elif kind == "N":
        for dr, dc in [(-2, -1), (-2, 1), (-1, -2), (-1, 2),
                       (1, -2), (1, 2), (2, -1), (2, 1)]:
            r1, c1 = row + dr, col + dc
            if in_bounds(r1, c1) and (empty(r1, c1) or enemy(r1, c1)):
                moves.append((r1, c1))

    elif kind in ("B", "R", "Q"):
        directions = []
        if kind in ("B", "Q"):
            directions += [(-1, -1), (-1, 1), (1, -1), (1, 1)]
        if kind in ("R", "Q"):
            directions += [(-1, 0), (1, 0), (0, -1), (0, 1)]
        for dr, dc in directions:
            r1, c1 = row + dr, col + dc
            while in_bounds(r1, c1):

```

```

        if empty(r1, c1):
            moves.append((r1, c1))
        elif enemy(r1, c1):
            moves.append((r1, c1))
            break
        else:
            break
        r1 += dr
        c1 += dc

elif kind == "K":
    for dr in (-1, 0, 1):
        for dc in (-1, 0, 1):
            if dr == 0 and dc == 0:
                continue
            r1, c1 = row + dr, col + dc
            if in_bounds(r1, c1) and (empty(r1, c1) or enemy(r1, c1)):
                moves.append((r1, c1))

# castling
if not piece.has_moved:
    row_back = 7 if color == WHITE else 0
    if row == row_back:
        # king side
        rook = board[row_back][7]
        if (rook and rook.kind == "R" and not rook.has_moved
            and empty(row_back, 5) and empty(row_back, 6)):
            moves.append((row_back, 6))
        # queen side
        rook = board[row_back][0]
        if (rook and rook.kind == "R" and not rook.has_moved
            and empty(row_back, 1) and empty(row_back, 2) and empty(row_back, 3)):
            moves.append((row_back, 2))

return moves

def find_king(board, color):
    for r in range(BOARD_SIZE):
        for c in range(BOARD_SIZE):
            p = board[r][c]
            if p and p.color == color and p.kind == "K":
                return r, c
    return None

def square_attacked(board, row, col, by_color):
    for r in range(BOARD_SIZE):
        for c in range(BOARD_SIZE):
            p = board[r][c]
            if p and p.color == by_color:
                # use simplified attack check (no en passant/castle needed for attack detection)
                for mr, mc in get_piece_moves(board, r, c, None):
                    if (mr, mc) == (row, col):
                        return True
    return False

def in_check(board, color):
    king_pos = find_king(board, color)
    if king_pos is None:
        return False
    enemy_color = BLACK if color == WHITE else WHITE
    return square_attacked(board, king_pos[0], king_pos[1], enemy_color)

def apply_move(board, src, dst, en_passant_target):
    """Mutates board, returns new en_passant_target."""
    sr, sc = src
    dr, dc = dst
    piece = board[sr][sc]
    new_ep_target = None

    # en passant capture
    if piece.kind == "P" and (dr, dc) == en_passant_target and board[dr][dc] is None:
        captured_row = sr # the pawn being captured sits on same row as source
        board[captured_row][dc] = None
    # castling: king moves two columns

```

```

if piece.kind == "K" and abs(dc - sc) == 2:
    row_back = sr
    if dc > sc: # king side
        rook = board[row_back][7]
        board[row_back][5] = rook
        board[row_back][7] = None
        rook.has_moved = True
    else: # queen side
        rook = board[row_back][0]
        board[row_back][3] = rook
        board[row_back][0] = None
        rook.has_moved = True

board[dr][dc] = piece
board[sr][sc] = None
piece.has_moved = True

# set en passant target if pawn double-stepped
if piece.kind == "P" and abs(dr - sr) == 2:
    new_ep_target = ((sr + dr) // 2, sc)

# promotion (auto-queen)
if piece.kind == "P" and (dr == 0 or dr == 7):
    piece.kind = "Q"

return new_ep_target

def legal_moves_for(board, row, col, en_passant_target):
    piece = board[row][col]
    if piece is None:
        return []
    legal = []
    for dst in get_piece_moves(board, row, col, en_passant_target):
        # simulate
        import copy
        board_copy = copy.deepcopy(board)
        apply_move(board_copy, (row, col), dst, en_passant_target)
        if not in_check(board_copy, piece.color):
            legal.append(dst)
    return legal

def main():
    board = make_initial_board()
    turn = WHITE
    en_passant_target = None

    print("Welcome to console Chess! Enter moves like: e2 e4")
    print("Type 'quit' to exit.\n")

    while True:
        print_board(board)
        if in_check(board, turn):
            print(f"*** {turn.capitalize()} is in check! ***")
        print(f"{turn.capitalize()}s move.")
        user_input = input("Move (from to): ").strip()
        if user_input.lower() in ("quit", "exit"):
            print("Goodbye!")
            break

        parts = user_input.split()
        if len(parts) != 2:
            print("Please enter two squares, e.g.: e2 e4")
            continue

        src = parse_square(parts[0])
        dst = parse_square(parts[1])
        if src is None or dst is None:
            print("Invalid square notation. Use letters a-h and numbers 1-8, e.g. e2")
            continue

        sr, sc = src
        piece = board[sr][sc]
        if piece is None:
            print("There's no piece on that square.")
            continue

```

```

if piece.color != turn:
    print(f"That's not your piece. It's {turn}'s turn.")
    continue

legal = legal_moves_for(board, sr, sc, en_passant_target)
if dst not in legal:
    print("Illegal move for that piece.")
    continue

en_passant_target = apply_move(board, src, dst, en_passant_target)

# checkmate / stalemate check for opponent
opponent = BLACK if turn == WHITE else WHITE
any_legal = False
for r in range(BOARD_SIZE):
    for c in range(BOARD_SIZE):
        p = board[r][c]
        if p and p.color == opponent:
            if legal_moves_for(board, r, c, en_passant_target):
                any_legal = True
                break
    if any_legal:
        break

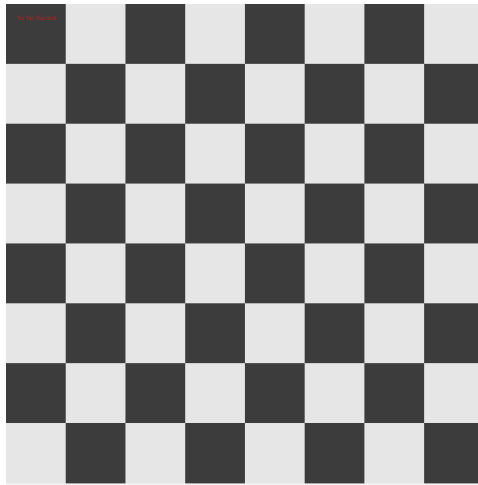
if not any_legal:
    print_board(board)
    if in_check(board, opponent):
        print(f"Checkmate! {turn.capitalize()} wins!")
    else:
        print("Stalemate! It's a draw.")
    break

turn = opponent

if __name__ == "__main__":
    main()

```

# Tic-Tac-Toe (tictactoe.py)



```
"""
Simple two-player console Tic-Tac-Toe.
Run with: python tictactoe.py

Players take turns entering a number 1-9 corresponding to the board position:

 1 | 2 | 3
 4 | 5 | 6
 7 | 8 | 9

Type 'quit' to exit.
"""

WIN_COMBOS = [
    (0, 1, 2), (3, 4, 5), (6, 7, 8), # rows
    (0, 3, 6), (1, 4, 7), (2, 5, 8), # columns
    (0, 4, 8), (2, 4, 6),             # diagonals
]

def print_board(board):
    print()
    for i in range(0, 9, 3):
        row = board[i:i + 3]
        print(" " + " | ".join(row))
        if i < 6:
            print("---+---+---")
    print()

def check_winner(board, mark):
    return any(all(board[i] == mark for i in combo) for combo in WIN_COMBOS)

def board_full(board):
    return all(cell != " " for cell in board)

def get_move(board, player):
    while True:
        choice = input(f"Player {player}, enter a position (1-9): ").strip()
        if choice.lower() in ("quit", "exit"):
            print("Goodbye!")
            exit()
        if not choice.isdigit() or not (1 <= int(choice) <= 9):
            print("Please enter a number between 1 and 9.")
            continue
        idx = int(choice) - 1
        if board[idx] != " ":
            print("That spot is already taken. Try again.")
            continue
        return idx
```

```

def main():
    board = [" "] * 9
    players = {"X": "X", "O": "O"}
    current = "X"

    print("Welcome to Tic-Tac-Toe!")
    print_board(["1", "2", "3", "4", "5", "6", "7", "8", "9"])

    while True:
        print_board(board)
        idx = get_move(board, current)
        board[idx] = current

        if check_winner(board, current):
            print_board(board)
            print(f"■ Player {current} wins!")
            break

        if board_full(board):
            print_board(board)
            print("It's a draw!")
            break

        current = "O" if current == "X" else "X"

if __name__ == "__main__":
    main()

```



## Appendix: Board Reference Images

